Programming - I, Lab 9: Strings, Lists and Loops

1. Read a line of space-separated integers and store the even and odd numbers in two separate lists. Print both lists.

Input	Output
`12 7 9 20 33 4`	`Even: [12, 20, 4]` `Odd: [7, 9, 33]`

2. Read a sentence and build a list of all the words that have more than 3 letters, in the order they appear. Print the list.

Input	Output
`Python is a fun language to learn`	`['Python', 'language', 'learn']`

3. The JKLU inter-hostel cricket league keeps a ball-by-ball record of every innings as a single line. Each ball is recorded as the runs scored on it (`0`, `1`, `2`, `3`, `4` or `6`), or as `W` if a wicket fell (no runs on that ball). An over has 6 balls, and the last over may be incomplete. The scorer wants the total runs, the number of wickets, the number of boundaries (balls on which 4 or 6 was scored), and a list of runs scored in each over.

   Input: A single line containing the ball-by-ball record, separated by spaces.

   Output: Print the total runs, wickets, boundaries and the list of runs per over, as shown in the samples.

   Constraints:
   1 ≤ number of balls ≤ 120

Sample Input	Sample Output

`1 4 0 6 W 2 1 0 4 W 6 1`	`Total Runs: 25`
	`Wickets: 2`
	`Boundaries: 4`
	`Runs per over: [13, 12]`

`0 0 1 W 2 6 4 1`	`Total Runs: 14`
	`Wickets: 1`
	`Boundaries: 2`
	`Runs per over: [9, 5]`

4. Read a list of integers and print the largest and the smallest number in it, without using `max()` or `min()`.

Input	Output
`12 45 7 30 19`	`Largest: 45` `Smallest: 7`

5. Read a sentence and print its longest word. If two words have the same length, print the one that comes first.

Input	Output
`Python makes coding fun`	`Python`

6. The campus canteen wants a quick billing program. The first line lists the items on the menu and the second line lists their prices in the same order. The third line lists the items a student ordered, and an item can appear more than once. Print one line for each item that was ordered, in menu order, showing the item, its quantity and its cost, followed by the total bill.

   Input: Line 1: menu items separated by spaces.
   Line 2: prices of the menu items, in the same order.
   Line 3: items ordered, separated by spaces.

   Output: For each ordered item, in menu order, print `item x quantity = cost`. Then print `Total: amount`.

   Constraints:
   1 ≤ number of menu items ≤ 20
   Every ordered item is on the menu. Item names are single lowercase words.

Sample Input	Sample Output

`samosa tea maggi sandwich`
`15 10 40 50`
`tea samosa tea maggi tea`
	`samosa x 1 = 15`
	`tea x 3 = 30`
	`maggi x 1 = 40`
	`Total: 85`

`chai coffee`
`12 25`
`coffee coffee`
	`coffee x 2 = 50`
	`Total: 50`

7. Read a sentence and print it with every word reversed, keeping the words in their original order.

Input	Output
`JKLU Jaipur rocks`	`ULKJ rupiaJ skcor`

8. Read a string and build a list of all the digits present in it, as integers, in the order they appear. Print the list and the sum of the digits.

Input	Output
`JKLU2026 Jaipur302017`	`[2, 0, 2, 6, 3, 0, 2, 0, 1, 7]` `Sum: 23`

9. The IT cell is creating college email IDs for newly admitted students. An email ID is the first name and last name in lowercase, joined by a dot, followed by `@jklu.edu.in`. If the same name has already been given an ID, the second student with that name gets the number 2 after the last name, the third gets 3, and so on. Names are compared without regard to case.

   Input: The first line contains N, the number of students. Each of the next N lines contains a first name and a last name.

   Output: Print the N email IDs, one per line, in the order the names were given.

   Constraints:
   1 ≤ N ≤ 50
   Each name has exactly two words.

Sample Input	**Sample Output**

`4`
`Rahul Verma`
`Priya Sharma`
`rahul verma`
`Rahul Verma`
	`rahul.verma@jklu.edu.in`
	`priya.sharma@jklu.edu.in`
	`rahul.verma2@jklu.edu.in`
	`rahul.verma3@jklu.edu.in`

10. Read a list of integers and move all the zeros to the end of the list, keeping the other numbers in their original order. Print the list.

Input	**Output**
`0 5 0 3 12 0 8`	`[5, 3, 12, 8, 0, 0, 0]`